Connecting a DHT22 sensor to a Raspberry Pi 4 is a classic maker project. It's perfect for getting data on temperature and humidity, but it can be a little finicky if the wiring or the code library isn't exactly right.
Here is a straightforward guide to getting it up and running safely.

1. The Hardware Setup

The DHT22 has four pins (though typically only three are used on pre-made module boards). If you have the bare sensor, you will need a **4.7kΩ to 10kΩ resistor** to act as a pull-up between the VCC and Data pins. If your DHT22 is already mounted on a small PCB (module), it likely has this resistor built-in.

Pinout Connections

Power down your Raspberry Pi before making these connections to avoid accidental shorts.

DHT22 Pin	Raspberry Pi 4 Pin	Notes
VCC (Power)	**Pin 1 or 17** (3.3V) or **Pin 2 or 4** (5V)	3.3V is generally recommended and safer for GPIO logic levels.
DATA (Signal)	**Pin 7** (GPIO 4)	You can use almost any free GPIO pin, but GPIO 4 is the standard default.
NC (Not Connected)	*Ignore*	Only present on bare 4-pin sensors; do not connect it to anything.
GND (Ground)	**Pin 9, 14, 20, or 25** (GND)	Any ground pin will work.

2. Preparing the Raspberry Pi

The old Adafruit DHT library has been deprecated for a while, so we will use the modern **CircuitPython Adafruit_Blinka** library, which allows Python to interact with the Pi's GPIO pins seamlessly.
Open your terminal and run the following commands to update your system and install the required Python packages:

```bash
# Update your package list
sudo apt update

# Install Python pip and libgpiod (required for modern GPIO access)
sudo apt install python3-pip libgpiod-dev

# Install the Adafruit DHT library
pip3 install adafruit-circuitpython-dht
```

⚠️ **Note on Python Virtual Environments:** If you are running a newer OS (like Raspberry Pi OS Bookworm), pip might give you an externally-managed-environment error. If that happens, create a virtual environment first by running python3 -m venv env, activate it with source env/bin/activate, and *then* run the pip3 install command.

3. The Python Code

Create a new Python file (e.g., dht22_test.py) using your favorite text editor:

```bash
nano dht22_test.py
```

Paste the following script into the file:

```python
import time
import board
import adafruit_dht

# Initialize the DHT22 on GPIO 4 (Pin 7)
# If you used a different GPIO pin, change board.D4 to board.D<your_pin>
dht_device = adafruit_dht.DHT22(board.D4)

print("Starting DHT22 reading... Press Ctrl+C to exit.\n")
```

```
while True:
    try:
        # Read temperature and humidity
        temperature_c = dht_device.temperature
        temperature_f = temperature_c * (9 / 5) + 32
        humidity = dht_device.humidity

        # Print the results
        print(f"Temp: {temperature_c:.1f}°C ({temperature_f:.1f}°F) | Humidity: {humidity:.1f}%")

    except RuntimeError as error:
        # DHT sensors are notoriously noisy; reading errors are common.
        # We just catch the error and keep trying.
        time.sleep(2.0)
        continue
    except Exception as error:
        dht_device.exit()
        raise error

    # The DHT22 requires a 2-second delay between readings
    time.sleep(2.0)
```

Save and exit (in Nano, press Ctrl+O, Enter, then Ctrl+X).

4. Running the Script

Execute your script from the terminal:

```
Bash
python3 dht22_test.py
```

Troubleshooting Tips

- **RuntimeError: A raw reading was not successful**: Don't panic if you see this occasionally. The DHT22 relies on precise timing that the Pi's operating system sometimes interrupts. The code is designed to ignore this and try again.
- **Constant Errors?** If you *never* get a successful reading, double-check your wiring. Ensure your Data pin matches the GPIO number in the code (remember, GPIO 4 is

physical Pin 7, not Pin 4). If using a bare sensor, ensure you didn't forget the pull-up resistor.